Spring 2026
Data Structures Section D20 — Lab

Topic

Introduction to Abstract Data Types: 1. AbstractArray 2. Stack

Objective

- Understand Abstract Data Types: AbstractArray & Stack
- Understand the use Global Functions on AbstractArray & Stack

Outcomes

1. Implement Functions on AbstractArray & Stack
2. Write Complete C++ Programs using AbstractArray & Stack only

Content

The following sections will be covered during this lab session.

1. **Basic Functions** using AbstractArray & Stack
2. **Complete Programs** for Given Problems using AbstractArray & Stack

Tasks

Students are required to complete the following tasks in lab timings.

For the tasks 1-2, create a private repository in your account on Github: **S26_DS_Array_Stack.** Upload codes with the names **Task_1**, **Task_2** and so on it.

Class Code Files to be Used in the Tasks

Must use the Class Code File as is. In case you need to change then provide reasons in the comments.

```
// AbstractArray.h
class AbstractArray
{
 public:
  AbstractArray(int _maxCapacity = 0);
  AbstractArray(const AbstractArray & AA);
  virtual void add(int v) = 0; // increments currentIndex and add value if not full
  virtual bool remove(int & value) = 0; // if not empty returns the value at the currentIndex and decrements
currentIndex
  virtual bool get(int index, int & value) = 0; // return the value at the index if index is valid
  virtual void insert(int index, int value) = 0; // inserts the value at the index if index is valid, replaces
existing value
  bool isEmpty( );
  bool isFull( );
 protected:
  int currentIndex;
  int * values;
```

```
  int maxCapacity;
};

 AbstractArray::AbstractArray(int _maxCapacity)
 {
   currentIndex = -1;
   maxCapacity = _maxCapacity;
   values = new int[maxCapacity]{0};
 }
 AbstractArray::AbstractArray(const AbstractArray & AA)
 {
   maxCapacity = AA.maxCapacity;
   values = new int[maxCapacity]{0};
   /*
   for(int i = 0; i < maxCapacity; i++)
     {
       values[i] = AA.values[i];
     }
   */
 }

bool AbstractArray::isEmpty( )
{
  return (currentIndex < 0);
}
bool AbstractArray::isFull( )
  {
  return (currentIndex == maxCapacity - 1);
}
```

```
// MyArray.h
class MyArray: public AbstractArray
{
 public:
  MyArray(int _currentIndex = 0): AbstractArray(_currentIndex){};
  MyArray(const MyArray & ma): AbstractArray(ma){};
  void add(int v);
  bool remove(int & v);
  bool get(int index, int & v);
  void insert(int index, int value);
};

void MyArray::add(int v)
{
  if(!isFull())
    {
      values[++currentIndex] = v;
      return;
    }
     cout << "Array is full\n";
}
```

```
bool MyArray::remove(int & v)
  {
    if(!isEmpty())
    {
      v = values[currentIndex--];
      return true;
    }
    return false;
}

bool MyArray::get(int index, int & v)
{
  if(index > 0 && index <= currentIndex)
    {
      v = values[index];
      return true;
    }
  return false;
}

void MyArray::insert(int index, int value)
{
  if(index > 0 && index <= currentIndex)
    {
      values[index] =  value;
      return;
    }
  cout << index << " is invalid\n";
}
```

```
// Stack.h
class Stack
{
 public:
  Stack(int _maxCapacity = 0);
  Stack(const Stack & AA);
  virtual void push(int v) = 0; // increments currentIndex and add value if not full
  virtual bool pop(int & value) = 0; // if not empty returns the value at the currentIndex and decrements
currentIndex
  bool isEmpty( );
  bool isFull( );
 protected:
  int currentIndex;
  int * values;
  int maxCapacity;
};

 Stack::Stack(int _maxCapacity)
 {
   currentIndex = -1;
   maxCapacity = _maxCapacity;
   values = new int[maxCapacity]{0};
```

```
 }
 Stack::Stack(const Stack & AA)
 {
   maxCapacity = AA.maxCapacity;
   values = new int[maxCapacity]{0};
 }

bool Stack::isEmpty( )
{
  return (currentIndex < 0);
}
bool Stack::isFull( )
  {
  return (currentIndex == maxCapacity - 1);
}
```

```
// MyStack.h
class MyStack: public Stack
{
 public:
  MyStack(int _currentIndex = 0): Stack(_currentIndex){};
  MyStack(const MyStack & ma): Stack(ma){};
  void push(int v);
  bool pop(int & v);
};

void MyStack::push(int v)
{
  if(!isFull())
    {
      values[++currentIndex] = v;
      return;
    }
     cout << "Array is full\n";
}

bool MyStack::pop(int & v)
  {
    if(!isEmpty())
    {
      v = values[currentIndex--];
      return true;
    }
    return false;
}
```

Task 1

Implement the code for the following functions, use only **AbstractArray** and no other data type **(except int, float, char and string)**. Test your code after adding function codes to the main.cpp (**must not change unless you can point out errors and describe in comments)** below and upload it to your repository**.** The name of the code on your repository should be **Task_1.cpp.** Include **AbstractArray.h and MyArray.h** in **Task_1.cpp**

```
// main.cpp
#include <iostream>
using namespace std;

#include "AbstractArray.h"
#include "MyArray.h"

void read(AbstractArray *, istream &); // reads 1 to 10 integers from the user
Void display (AbstractArray *, ostream &); // displays values in the AbstractArray
void copy(AbstractArray * src, AbstractArray * dst); // copy src AbstractArray to dst AbstractArray
void insert(AbstractArray * src, AbstractArray * dst, int pos); // insert src AbstractArray to dst AbstractArray at the given pos
void shiftLeft(AbstractArray * aa, int pos); // shift left AbstractArray aa from the given pos
void shiftRight(AbstractArray * aa, int pos); // shift right AbstractArray aa from the given pos

void stats(AbstractArray * AA, int max, float average);// calculate maimum and average value of AA

int main()
{
  AbstractArray * A = new AbstractArray(MAX_SIZE);
  int n = 0;
  cout << "please select a number between 1 and " << MAX_SIZE << ": ";
  cin >> n;
  read(A, cin, n);
  cout << "Array A Values: ";
  display(A, cout);
  AbstractArray * B = new AbstractArray(MAX_SIZE);
  copy(A, B);
  cout << "Array B Values: ";
  display(B, cout);
  AbstractArray * C = new AbstractArray(MAX_SIZE);
  C.add(9);
  C.add(7);
  C.add(5);
  insert(A, C, 2);
  cout << "Array C Values: ";
  display(C, cout);
  shiftLeft(C, 2);
  cout << "Array C Values: ";
  display(C, cout);
  shiftRight(C, 2);
  cout << "Array C Values: ";
```

```
  display(C, cout);
  int mx = 0;
  float ag = 0.0;
  stats(C, mx, ag);
  cout << "Array C Values: ";
  display(C, cout);
  cout << "Average is " << ag << " maximum is " << mx << endl;
  return 0;
}
```

Task 2

Implement the code for the following functions, use only **Stack (except int, float, char and string)** and no other data type. Test your code after adding function codes to the main.cpp (**must not change unless you can point out errors**) below and upload it to your repository. **Any change of function prototype must be justified with a reason in the comments.** The name of the code on your repository should be **Task_2.cpp**. Include Stack.h and MyStack.h in Task_2.cpp

```
#include <iostream>
using namespace std;

#include "Stack.h"
#include "MyStack.h"

void read(Stack *, istream &); // reads 1 to 10 integers from the user
Void display (Stack *, ostream &); // displays values in the Stack
void copy(Stack * src, Stack * dst); // copy src Stack to dst Stack
void insert(Stack * src, Stack * dst, int pos); // insert src Stack to dst
Stack at the given pos
void shiftUp(Stack * s, int pos); // shift Stack s Up from the given pos
void shiftDown(Stack * s, int pos); // shift Stacks Down from the given pos

void stats(Stack * s, int max, float average);// calculate maimum and
average value stored in s

int main()
{
  Stack * A = new Stack(MAX_SIZE);
  int n = 0;
  cout << "please select a number between 1 and " << MAX_SIZE << ": ";
  cin >> n;
  read(A, cin, n);
  cout << "Stack A Values: ";
  display(A, cout);
  Stack * B = new Stack(MAX_SIZE);
  copy(A, B);
  cout << "Stack B Values: ";
  display(B, cout);
  Stack * C = new Stack(MAX_SIZE);
  C.push(9);
  C.push(7);
  C.push(5);
  insert(A, C, 2);
  cout << "Stack C Values: ";
```

```
  display(C, cout);
  shiftLeft(C, 2);
  cout << "Stack C Values: ";
  display(C, cout);
  shiftRight(C, 2);
  cout << "Stack C Values: ";
  display(C, cout);
  int mx = 0;
  float ag = 0.0;
  stats(C, mx, ag);
  cout << "Stack C Values: ";
  display(C, cout);
  cout << "Average is " << ag << " maximum is " << mx << endl;
  return 0;
}
```

Task 3

Implement complete C++ program for the following Problem and upload it to your repository**. You can only use Stack and no other Data Types (except int, float, char and string). Must reuse function codes from previous Tasks.** The name of the code on your repository should be **Task_3.cpp.** Include **Stack.h and Stack.h** in **Task_3.cpp**

The program should first read 10 integers from the use in a Stack. Then checks and displays if each Stack value for: (**must use a separate global function for each task)**

1. Prime
2. Perfect Square
3. Palindrome

Next check if the values stored in the Stack form a Palindrome i.e. same in each direction e.g. 123 456 539 456 123 forms a Palindrome but 123 456 789 345 does not.

Task 4

Implement complete C++ program for the following Problem and upload it to your repository**. You can only use Stack and no other Data Types (except int, float, char and string). Must reuse function codes from previous Tasks** The name of the code on your repository should be **Task_4.cpp.** Include **Stack.h and MyStack.h** in **Task_4.cpp**

The **program reads an English paragraph** from a file "para.txt" and **of maximum 100 words).** Then **do these tasks:** (**must use a separate global function for each task)**

1. **Copy the words** stored in the Stack **in reverse order** in **another Stack and display** the Stack **reverse copy.**
2. **Remove duplicates** from the **Stack** with **the** reverse copy **and display**
3. **Counts the frequency of each word and store** in an **int Stack. Display each word once and its frequency**

Task 5

Binary search is another searching algorithm used to search a specific value (or index of value) from the *sorted array*. In binary search, we first compare the *value to be searched* with the item in the middle position of the array. If there's a match, we can return immediately. If the key is less than the middle key, then the item sought must lie in the lower half of the array; if it's greater than the item sought must lie in the upper half of the array. So we repeat the procedure on the lower (or upper) half of the array. The pseudocode of the task is given below:

```
Input: A Sorted array, A of N elements and value to be searched
Output: Index of searched element or -1 if not found

low = 0, high = N-1;
While (low <= high)
   mid = (low + high)/2;
   If ( A[mid] == Value )
     Return mid;
   Else-if ( A[mid] < Value )
     low = mid + 1;
   Else
     high = mid - 1;
   End-if
```

```
End-While
return -1;
```

Implement the code for binary search using **<u>templates</u>**. Test your code using the main function below and upload it to your repository**.** The name of the code on your repository should be **Task_5.cpp**

```
int main() {
    // Test with an integer array (sorted) of size 5
    int intArray[5] = {11, 12, 22, 25, 64};
    int intKey = 22;
    int intIndex = binarySearch(intArray, intKey);
    printSearchResult(intIndex, intKey);

    // Test with a float array (sorted) of size 4
    float floatArray[4] = {0.57, 1.62, 2.71, 3.14};
    float floatKey = 2.71;
    int floatIndex = binarySearch(floatArray, floatKey);
    printSearchResult(floatIndex, floatKey);

    // Test with a string array (sorted) of size 4
    string stringArray[4] = {"apple", "banana", "grape", "orange"};
    string stringKey = "grape";
    int stringIndex = binarySearch(stringArray, stringKey);
    printSearchResult(stringIndex, stringKey);

    return 0;
}
```